Object-oriented Programming

Week 13| Lecture 1

Stream

- C++ Input/Output implement concept of stream . Stream is the sequences of bytes or flow of data , which acts as a source from which the input data can be obtained by a program or a destination to which the output data can be sent by the program.

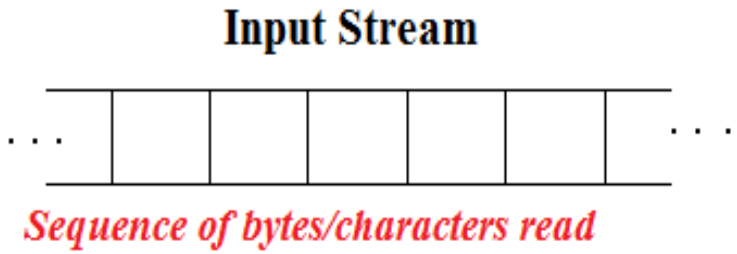
Stream

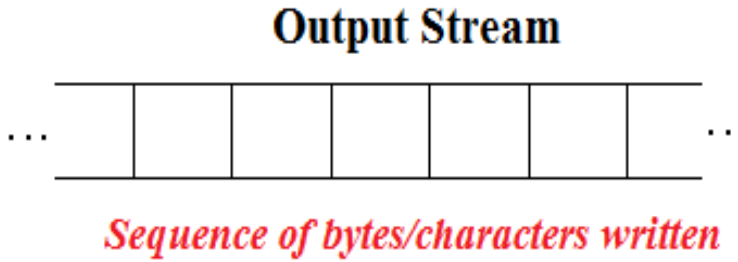
- A stream is an abstraction of a device where input/output operations are performed. You can represent a stream as either a destination or a source of characters of indefinite length.
- When we open a stream, one end is always attached to the program that opens the stream and the other end is attached to a file, which is accessed by its name.

Stream

- **Input Stream :**
- It is flow of data bytes from a device (e.g Keyboard , disk drive) to main memory (when we read/take file's data into a program variable)
- **Output Stream :**
- It is flow of data bytes from main memory (i.e program) to a device(when we store/write variable's data into a file)

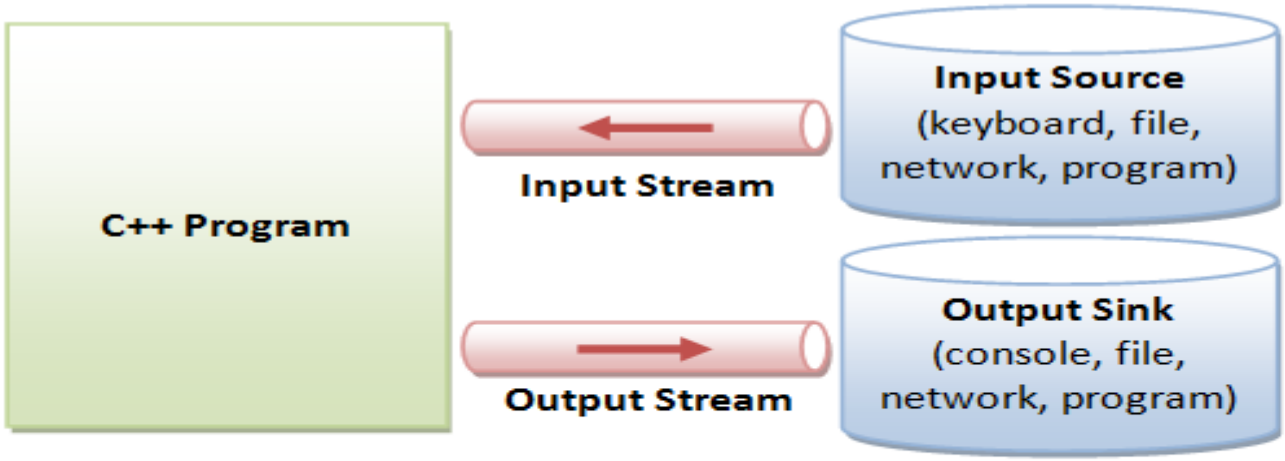
Stream


Input Stream
Sequence of bytes/characters read


Output Stream
Sequence of bytes/characters written

Stream

- In input operations, Data bytes flow from an *input source* (such as keyboard, file, network or another program) into the program.
- In output operations, data bytes flow from the program to an *output sink* (such as console, file, network or another program). Streams acts as an intermediaries between the programs and the actual IO devices



Internal Data Formats:

- Text: `char`, `wchar_t`
- `int`, `float`, `double`, etc.

External Data Formats:

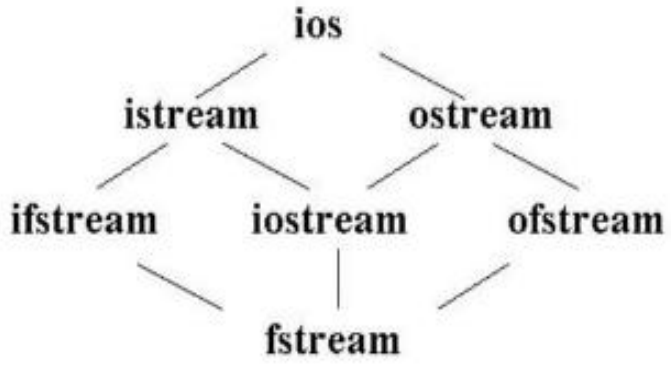
- Text in various encodings (US-ASCII, ISO-8859-1, UCS-2, UTF-8, UTF-16, UTF-16BE, UTF16-LE, etc.)
- Binary (raw bytes)

- Files store data permanently in a storage device. With file handling, the output from a program can be stored in a file. Various operations can be performed on the data while in the file.
- **Using file handling we can store our data in Secondary memory (Hard disk).**

Why use File Handling in C++:-

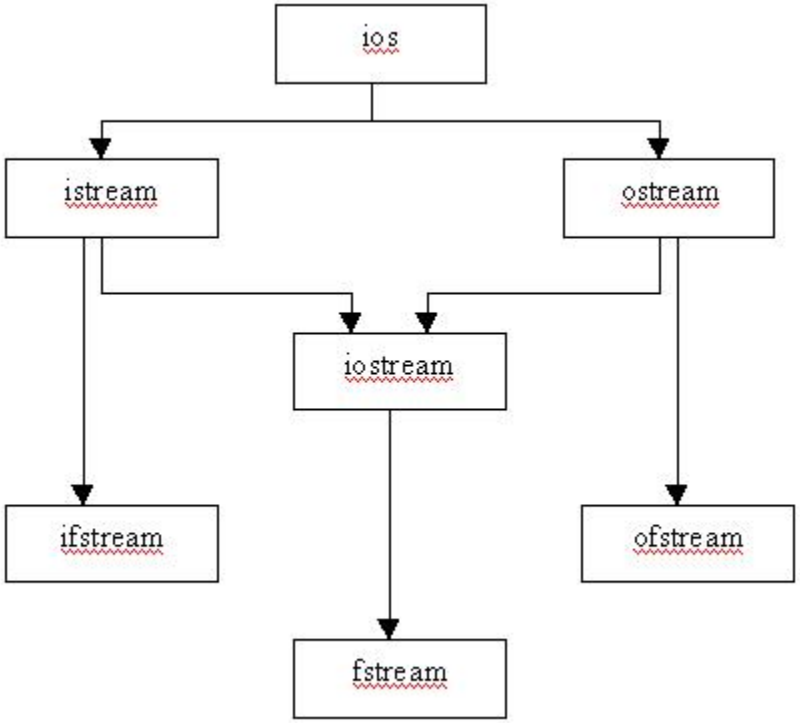
- For permanet storage.
- The transfer of input - data or output - data from one computer to another can be easily done by using files.

Classes for stream input and output



- ios is the base class.
- istream and ostream inherit from ios
- ifstream inherits from istream (and ios)
- ofstream inherits from ostream (and ios)
- iostream inherits from istream and ostream (& ios)
- fstream inherits from ifstream, iostream, and ofstream

Stream Class Hierarchy



File I/O

- To perform file I/O, you must include the header **<fstream> in your program**

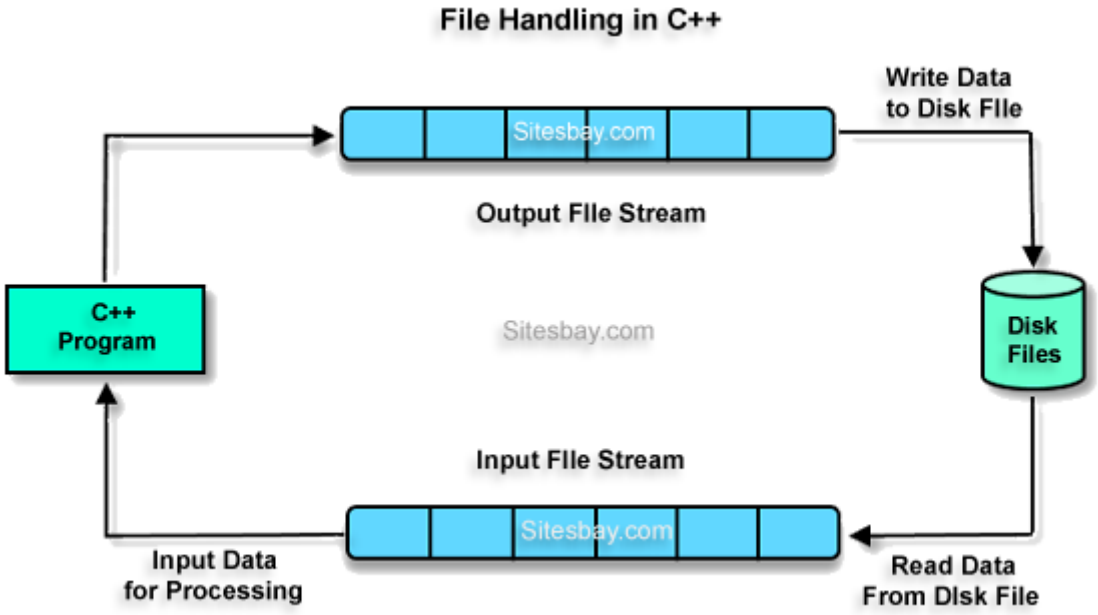
- It defines several classes, including **ifstream, ofstream, and fstream**

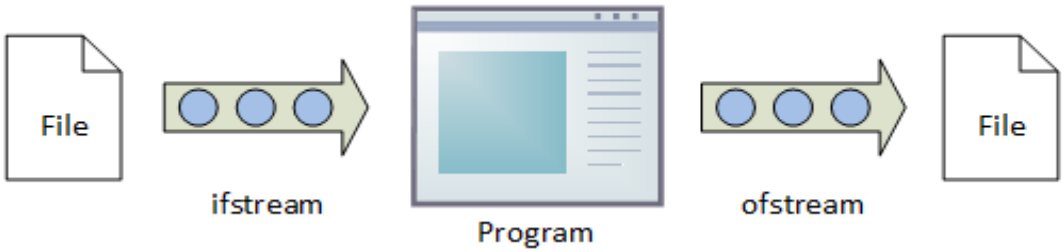
- C++ views each file as a sequence of bytes

File Stream

- There are three types of file streams: input, output, and input/output

```
ifstream    in;     // input stream object
ofstream  out;  // output stream object
fstream    io;     // input/output stream object
```


File Handling in C++
Write Data
to Disk File
Sitesbay.com
Output File Stream
C++
Program
Sitesbay.com
Disk
Files
Input File Stream
Sitesbay.com
Input Data
for Processing
Read Data
From Disk File


File
ifstream
Program
ofstream
File

Ofstream predefined methods

- This class is used to prepare an object which performs write operations on file.
- its predefined methods are
- open()
- write()
- put()
- seekp()
- tellp()
- close()

ifstream predefined methods

- it is used to prepare an object which performs read operations on file.
- Its predefined methods are
- open()
- read()
- get()
- getline()
- seekg()
- tellg()
- close()

fstream predefined methods

- **It performs both read and write operations of file**

Reading from a file

//open a file for reading using constructor
ifstream in(“myfile”);

//open a text file for reading
ifstream in(“myfile.txt”);

if for some reason, file cannot be opened then ifstream object has the value ***false***

Alternate Syntax

- An alternate way to open a file for read/write is by using open() function

```
ofstream out;
out.open("test", ios::out);
```

ios Modes

`ios::in`	Open file for input
`ios::out`	Open file for output
`ios::app`	Append data to the end of the output file (file-pointer repositioning commands are ignored, forcing all output to take place at the end of the file)
`ios::ate`	Open the file at the end of the data (allows the file-pointer to be repositioning within the file)
`ios::trunc`	Truncates or discards the current contents of existing files
`ios::binary`	Opens the file in binary mode (without this mask, the file is opened in text mode by default)

class	default mode parameter
`ofstream`	ios::out
`ifstream`	ios::in
`fstream`	ios::in | ios::out

Prototypes for the stream constructors and open functions

```
ifstream
    ifstream (file_name, openmode mode = ios::in);
    open(file_name, openmode mode = ios::in);

ofstream
    ofstream (file_name, openmode mode = ios::out);
    open(file_name, openmode mode = ios::out);

fstream
    fstream (file_name, openmode mode = ios::in | ios::out);
    open(file_name, openmode mode = ios::in | ios::out);
```

Prototypes for the stream constructors and open functions. The function prototypes show the default open modes for the different kinds of file stream classes.

ORing

All these flags can be combined using the bitwise operator OR (|).

```
ofstream myfile;
myfile.open ("example.bin", ios::out | ios::app );
```

Closing a file

- To close a file, we can use the member function **close( )**

Example:

mystream.close();

Opening And Closing Files

```
ifstream input(file_name);
// use file
// "input" closed by destructor
```

```
ifstream input;
while (....)
{
        input.open(file_name);
        //use file
        input.close();
}
```

File open using constructor method

```
#include <iostream>
#include <fstream>
using namespace std;
int main(){
    ofstream f("XYZ");
    f << "hello";
    f.close();
}
```

Multiple File open using open method with same filestream

```
#include <iostream>
#include <fstream>
using namespace std;
int main(){
        ofstream f;
        f.open("file1");
        f << "hello";
        f.close();//without close compiler implicitly close the file
//       second file through the same file stream
        f.open("file2")    ;
        f<<"i am a student";
        f.close();
}
```

formatted and unformatted IO

- C++ provides both the formatted and unformatted IO functions.
- In formatted or high-level IO, bytes are grouped and converted to types such as int, double, string or user-defined types.
- In unformatted or low-level IO, bytes are treated as raw bytes and unconverted.
- Formatted IO operations are supported via overloading the stream insertion (<<) and stream extraction (>>) operators,

- The << and >> operators are overloaded to handle fundamental types (such as int and double), and classes (such as string). You can also overload these operators for your own user-defined types.

Types of I/O

- We can perform either formatted or unformatted I/O with file stream
- Formatted output is carried out on streams via the stream insertion << and stream extraction >> operators
- Character translations are performed between console window and files

Formatted I/O

- Formatted I/O can be performed by using extraction **>>** and insertion **<<** operators
- All information is stored in the file in the same format as it would be displayed on the screen
- When reading text files using the **>>** operator, certain character translations occur. For example, white-space characters are omitted

- Formatted output converts the internal binary representation of the data to ASCII characters which are written to the output file.
  Formatted input reads characters from the input file and converts them to internal form.

Advantages and Disadvantages of Formatted I/O

- Formatted input/output is very portable. It is a simple process to move formatted data files to various computers, even computers running different operating systems, as long as they all use the ASCII character set.
- Formatted files are human readable and can be typed to the terminal screen or edited with a text editor.

To write to the file, use the insertion operator (<<)

- ```
  #include <iostream>
  #include <fstream>
  using namespace std;

  int main() {
    // Create and open a text file
    ofstream MyFile("filename.txt");

    // Write to the file using insertion operator
    MyFile << "Files can be tricky, but it is fun enough!";

    // Close the file
    MyFile.close();
  }
  ```
  ```

Validate the file before trying to access

Method 1:

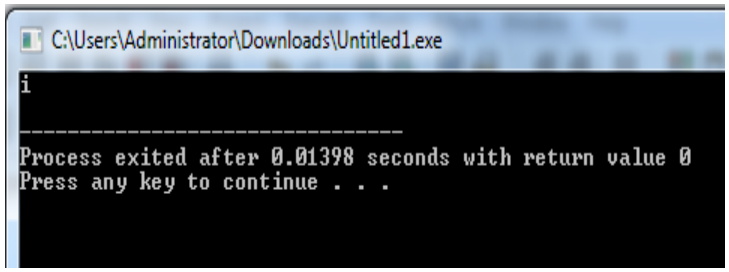
```
By checking the stream variable;
If ( ! Mystream)
{
Cout << "Cannot open file.\n ";
}
```

Method 2:

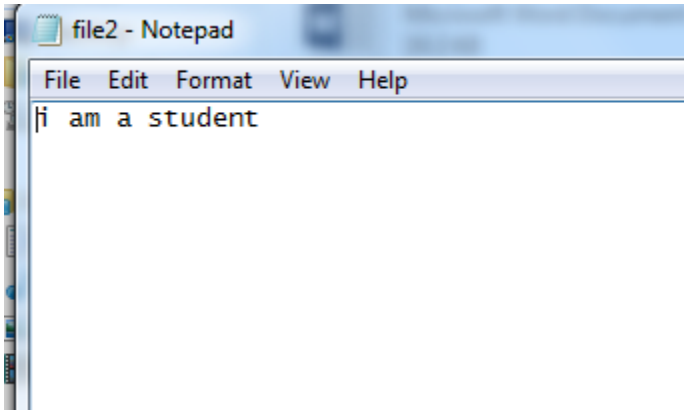
```
By using bool is_open() function.
If ( ! Mystream.is_open()) {
Cout << "File is not open.\n ";
}
```

Example for reading file

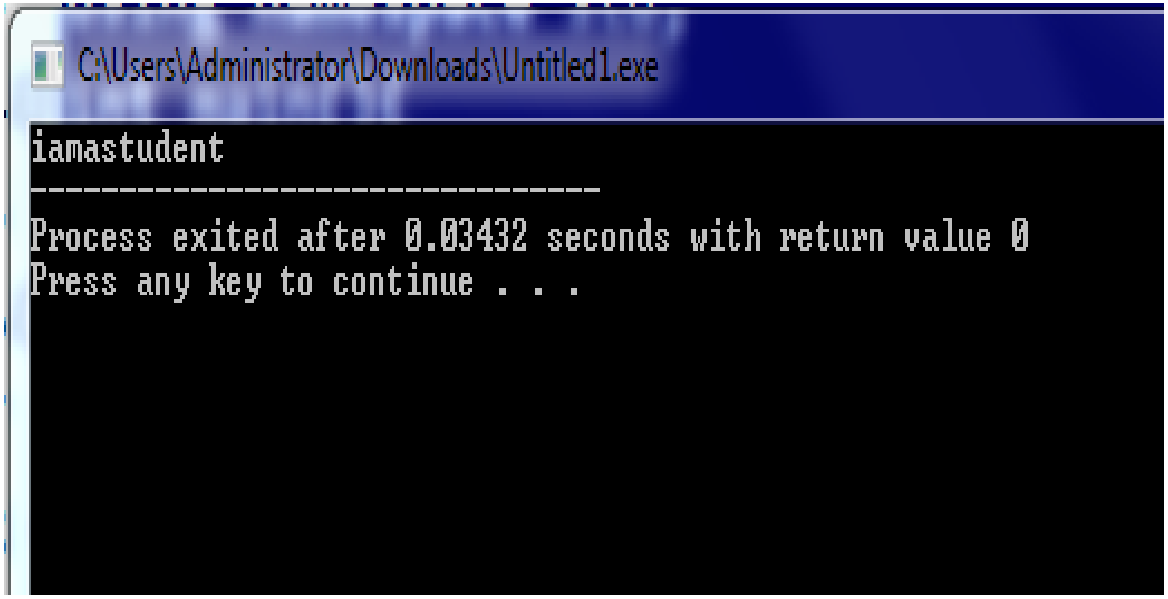
```
#include <iostream>
#include <fstream>
using namespace std;
int main(){
    ifstream in("File2");
    if(!in){
     cout << "Failed to open file." << endl;
      return 1; }
    char c[20];
      in >> c ;
      cout << c << endl;
    in.close();
    return 0;}
```


C:\Users\Administrator\Downloads\Untitled1.exe
i

Process exited after 0.01398 seconds with return value 0
Press any key to continue . . .


file2 - Notepad
File Edit Format View Help
i am a student

```
#include <iostream>
#include <fstream>
using namespace std;
int main(){
        ifstream in("File2");
        if(!in){
         cout << "Failed to open file." << endl;
          return 1; }
        char c[20];
        while(!in.eof()){
          in >> c ;
          cout << c ;}
        in.close();
        return 0;}
```


C:\Users\Administrator\Downloads\Untitled1.exe
iamastudent

Process exited after 0.03432 seconds with return value 0
Press any key to continue . . .

Writing to a file

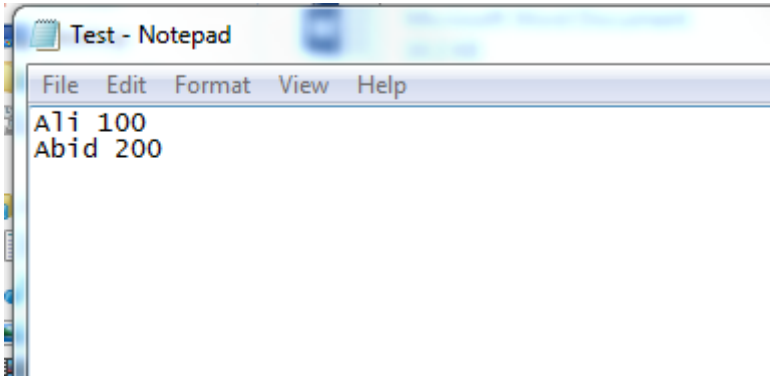
```
// open a file for writing
ofstream out("myfile");

// open a text file for writing
ofstream out("myfile.txt");
```

- If file does not already exist then it is created
- If the file cannot be opened or created, then ofstream object has the value ***false***

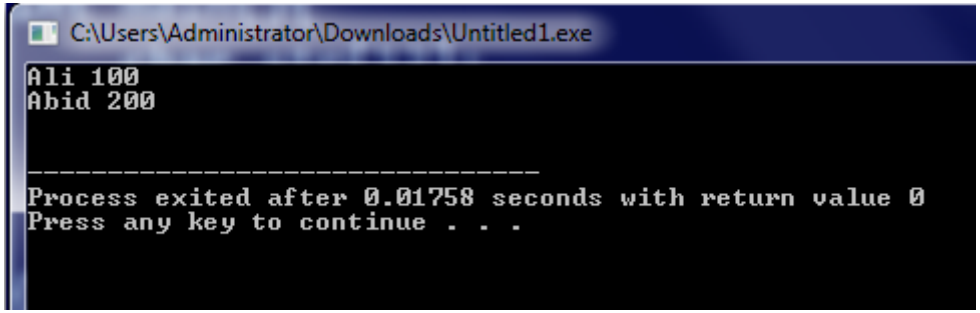
Example

```
#include <iostream>
#include <fstream>
using namespace std;
int main()
{
    ofstream out("Test.txt");
if(!out){
cout << "File creation failed";     return 1;}
out << "Ali " << 100 << endl;
out << "Abid " << 200 << endl;
out.close();return 0;
}
```


Test - Notepad
File Edit Format View Help
Ali 100
Abid 200

File Read through char array

```
#include <iostream>
#include <fstream>
using namespace std;
int main(){
        char str[12];
        ifstream f;
        f.open("Test.txt");
        while(f) //reading through file object
{
                f.getline(str,10);
                cout<<str<< endl;          }
        f.close();}
```


C:\Users\Administrator\Downloads\Untitled1.exe
Ali 100
Abid 200

Process exited after 0.01758 seconds with return value 0
Press any key to continue . . .

File Read through string object

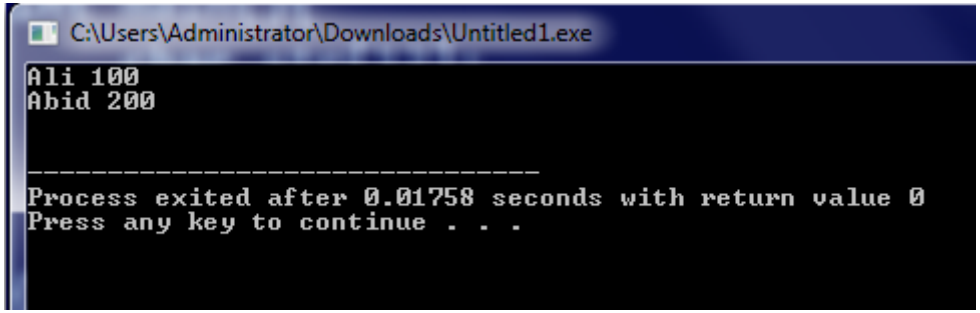
```
#include <iostream>
#include <fstream>
using namespace std;
int main(){
        string str;
        ifstream f;
        f.open("Test.txt");
        while(f){
                getline(f,str);
                cout<<str<<endl;       }
        f.close();}
```

File Read through string object

```
#include <iostream>
#include <fstream>
using namespace std;
int main(){
        string str;
        ifstream f;
        f.open("abc.txt");
        while(getline(f,str)){

                cout<<str<<endl;
        }

        f.close();
}
```


C:\Users\Administrator\Downloads\Untitled1.exe
Ali 100
Abid 200

Process exited after 0.01758 seconds with return value 0
Press any key to continue . . .

File read and write

```
#include <iostream>
#include <fstream>
using namespace std;
int main(){
        fstream file;
        file.open("smple.txt",ios::out);

        if(!file){
                cout<<"error";
                return 0;
        }
        cout <<"File created" << endl;
        file << "hi i am";
        file.close();}
```

File read and write

```
file.open("smple.txt",ios::in);
        if(!file){
                cout<<"error";
                return 0;
        }
        char ch[40];
        cout <<"contents are: ";
        while(!file.eof()) //reading through eof
{
                //file>>ch; white spaces omitted
                file.getline(ch,40);
                cout<<ch ;
        }
        file.close();
        return 0;
}
```

File read write w/o closing

```
#include <iostream>
#include <fstream>
using namespace std;
int main() {
   fstream f;
   f.open("oop", ios::in | ios::out | ios::ate);
   if (!f) {
      cout << "Failed to open file.";
      return 1;
   }
   f << "hello oop class. ";

   f.seekg(0);
   string s;
   while (getline(f, s)) {
      cout << s << endl;
   }

   f.close();
   return 0;
}
```

Unformatted I/O

- When we need to store unformatted (raw) binary data (not text) in a file, we can make use of the following set of functions
- When performing binary operations on a file, we open it using the **ios::binary** mode specifier
- Although unformatted file functions can work on text files, some character translations may still occur

Unformatted I/O

- Unformatted Input/Output is the most basic form of input/output. Unformatted input/output transfers the internal binary representation of the data directly between memory and the file

Unformatted I/O

- The unformatted output functions (e.g., put(), write()) outputs the bytes as they are, without format conversion.

- In unformatting input, such as get(), getline(), read(), it reads the characters as they are, without conversion.

Advantages and Disadvantages of Unformatted I/O

- Unformatted input/output is the simplest and most efficient form of input/output. It is usually the most compact way to store data. Unformatted input/output is the least portable form of input/output. Unformatted data files can only be moved easily to and from computers that share the same internal data representation.
- Unformatted input/output is not directly human readable, so you cannot type it out on a terminal screen or edit it with a text editor.

Get/Put Functions

- The functions get() and put() reads and writes a single character to a file, respectively

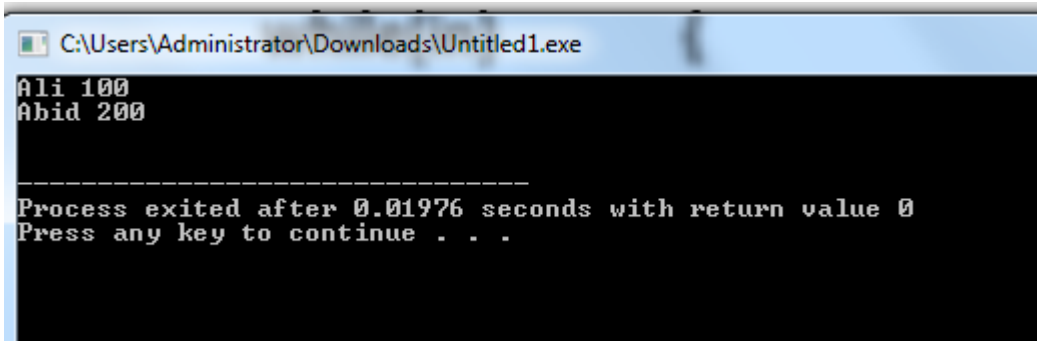
get() Function

- istream &get(char &*ch);*

- The **get( ) function reads a single character from the invoking stream and puts that** value in *ch. It returns a reference to the stream*

Example

```
#include <iostream>
#include <fstream>
using namespace std;
int main(){
    ifstream in("Test.txt");
        if(!in)    {
        cout << "Failed to open file" << endl;
        return 1;          }
        char c;
        while(in)          {
        in.get(c);
        cout << c;         }
    return 0;
}
```


C:\Users\Administrator\Downloads\Untitled1.exe
Ali 100
Abid 200

Process exited after 0.01976 seconds with return value 0
Press any key to continue . . .

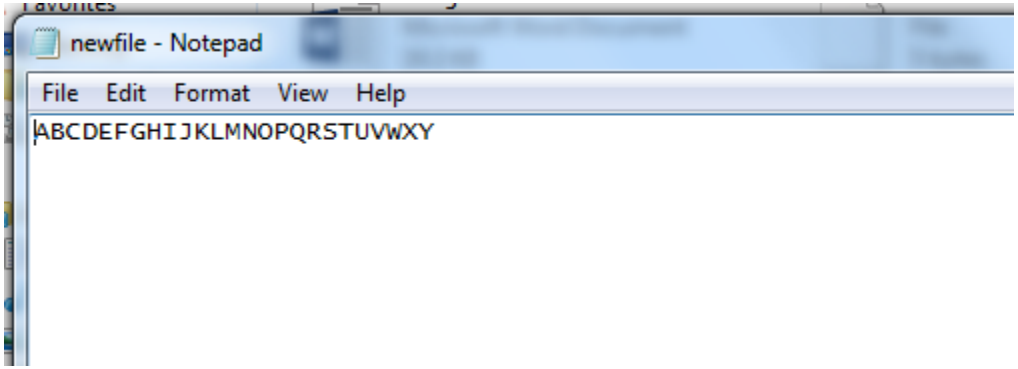
put() Function

- ostream &put(char *ch);

- *The **put( ) function writes ch to the*** stream and returns a reference to the stream

Example

```
#include <iostream>
#include <fstream>
using namespace std;
int main(){
ofstream o("newfile.txt");
	if(!o){
		cout << "Failed to open file" << endl;
		return 1;	}
		for(int i = 65; i < 90; i++){
		o.put(i);}
		o.close();
	return 0;
}
```


newfile - Notepad
File Edit Format View Help
ABCDEFGHIJKLMNOPQRSTUVWXY

get() and put() are for single characters, not full objects
These functions are for low-level character input/output:

Function	Purpose
get()	Reads a **single character**
put()	Writes a **single character**

Read/Write Functions

- The functions **read()** and **write()** are similar to get() and put() except that we can read and write entire blocks of bytes (e.g. character array) in a file

istream &read(char **buf, int num);*

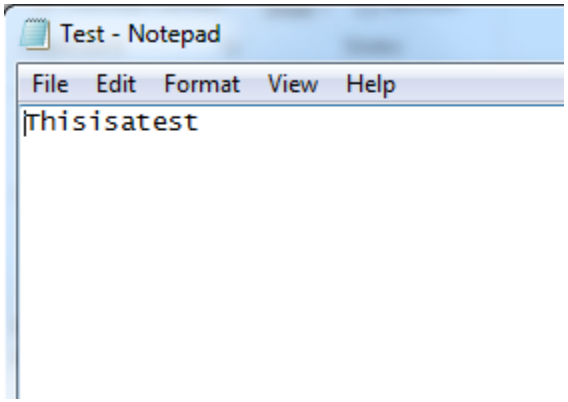
ostream &write(const char **buf, int num);*

Example

```
// Writing block (array) of characters
char chW[20] = {'T', 'h', 'i', 's', 'i', 's', 'a', 't','e','s','t'};
o.write(chW, 20);

// Reading block (array) of characters
char chR[20];
i.read(chR, 20);
for(int i = 0; i < 20; i++)
{
	cout << chR[i];
}
```

```
#include <iostream>
#include <fstream>
using namespace std;
int main(){
fstream in("Test.txt");
                char chW[20] = {'T', 'h', 'i', 's', 'i', 's', 'a',
't','e','s','t'};
        in.write(chW, 20);
        return 0;
}
```



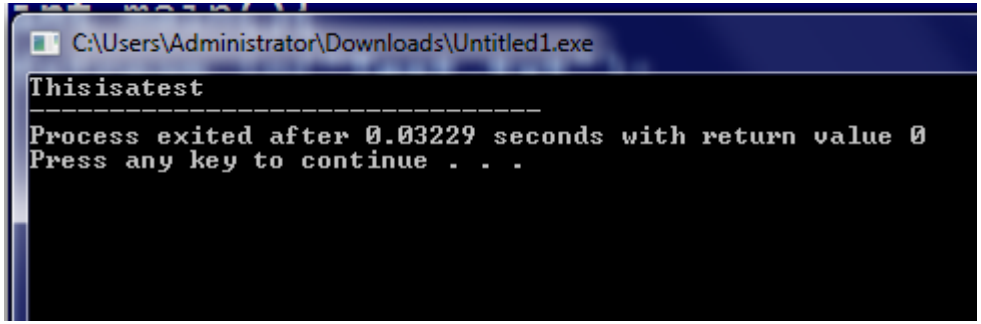
```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main() {
    ofstream out("Test.txt", ios::binary);

    string msg = "This is a test";
    out.write(msg.c_str(), msg.size());  // write raw bytes of the string

    out.close();
    return 0;
}
```

```
#include <iostream>
#include <fstream>
using namespace std;
int main(){
fstream in("Test.txt");
                char chR[20];
        in.read(chR, 20);
        for(int i = 0; i < 20; i++){
                cout << chR[i];}
        return 0;
}
```



eof() Function

bool eof( );

- You can detect when the end of the file is reached by using the member function **eof( )**

- It returns *true* when the end of the file has been reached; otherwise it returns *false*

Example 1

```
int main()
{

	ifstream i("myfile.txt");

	if(!i)
	{
		cout << "Failed to open file" << endl;
		return 1;
	}

	char c;

	for(int x = 0; x < 50; x++)
	{
		i.get(c);
		cout << c;
	}

	return 0;
}
```

myfile.txt contains text:
“Hello”

Output:
Hellooooooo

Example 2

```
int main()
{

	ifstream in("myfile.txt");

	if(!in)
	{
		cout << "Failed to open file" << endl;
		return 1;
	}

	char c;
	for(int x = 0; x < 10; x++)
	{
		if(in.eof())
			break;

		in.get(c);
		cout << c;
	}

	return 0;
}
```

flawed behavior

- Unfortunately, the behavior of the eof( ) function is not as straightforward as we might expect. The function does not actually test the file to see if there is more data to read. Instead, the eof( ) function returns the current value of the eof-bit (one of the stream's state flags), which is set by the last *read* function to run. That means that the value that eof( ) returns depends on the outcome of a different, previously run, function. This unexpected behavior is usually only a problem when reading single characters from a file - functions reading more complex data generally set the eof-bit as a part of the read operation.

flawed behavior

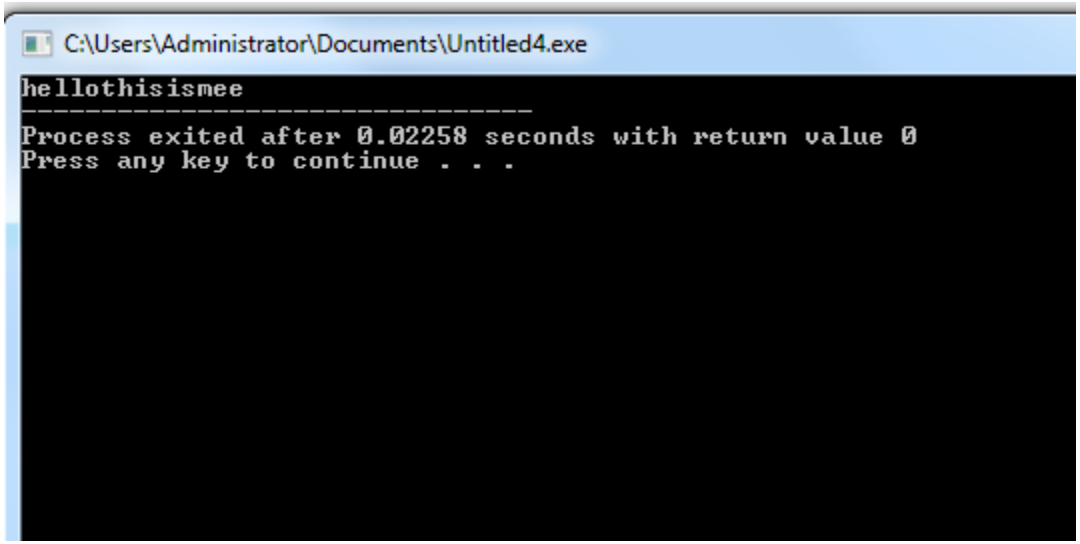
```
#include <iostream>
#include <fstream>
using namespace std;

int main()
{
        ifstream in("my_file");
        char c;

        while (!in.eof())
        {
                in >> c;
                cout  << c ;
        }

        return 0;
}
```

Duplication of last char

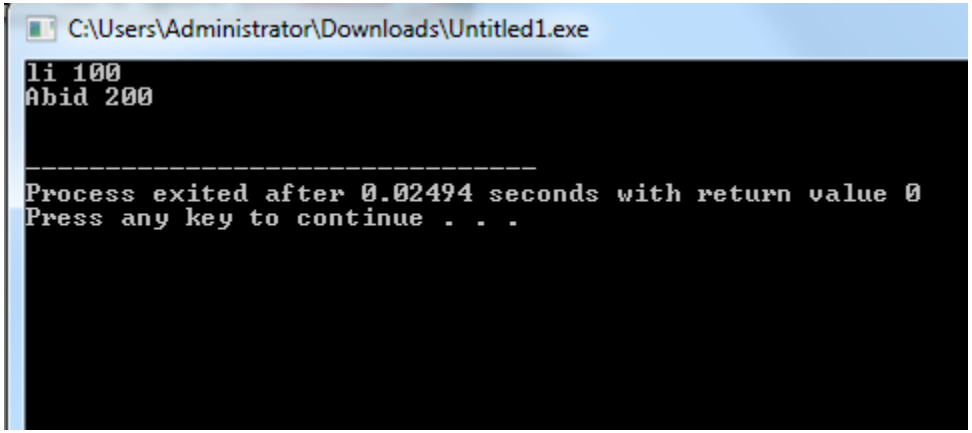


ignore() Function

istream &ignore(int *num_of_char*, **char** *delimiter*)

- It reads and discards characters until either *num characters have been ignored (1 by* default) or the character specified by *delim is encountered (**EOF by default)***

```
#include <iostream>
#include <fstream>
using namespace std;
int main(){
ifstream in("Test.txt");
                        if(!in)       {
                        cout << "Failed to open file" << endl;
                        return 1;  }
            char c;
            in.ignore();// 1 character by default
            while(in)  {
                        in.get(c);
                        cout << c;}
            return 0;
}
```


C:\Users\Administrator\Downloads\Untitled1.exe
li 100
Abid 200

Process exited after 0.02494 seconds with return value 0
Press any key to continue . . .

Example

```
int main()
{

	ifstream in("myfile.txt");

	if(!in)
	{
		cout << "Failed to open file" << endl;
		return 1;
	}

	char c;

	in.ignore(5, ' ');

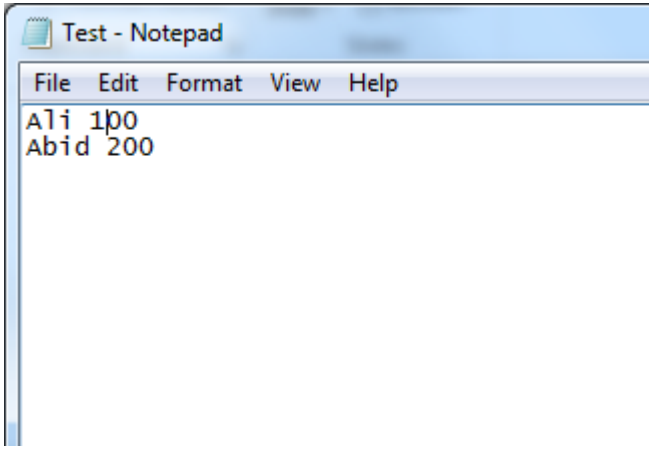
	while(in)
	{
		in.get(c);
		cout << c;
	}

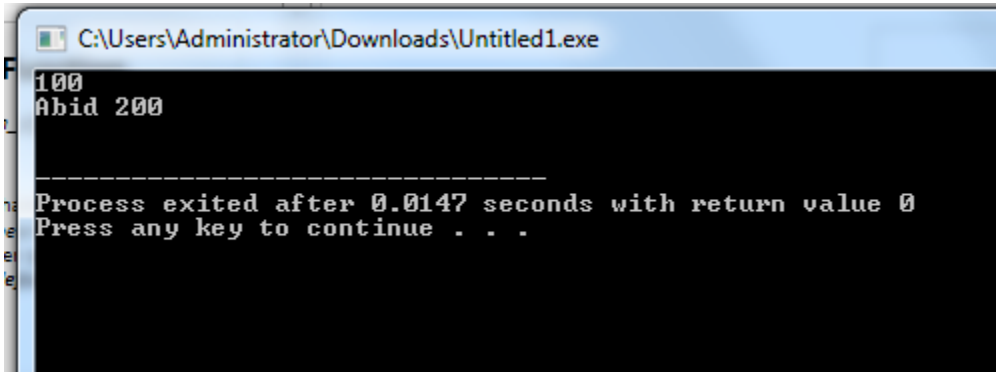
	return 0;
}
```

myfile.txt contains text:
“Hello this is test”

Output:
this is testt

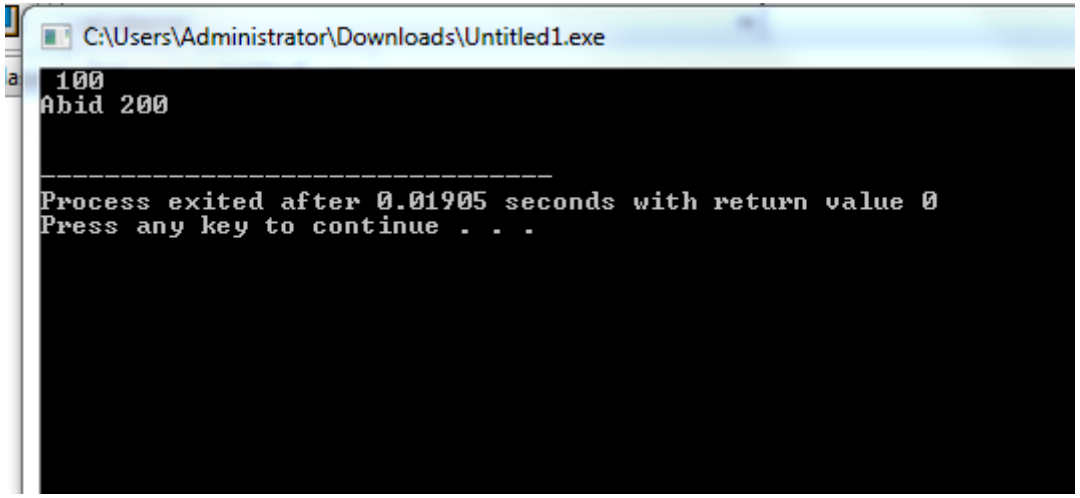
```
#include <iostream>
#include <fstream>
using namespace std;
int main(){
ifstream in("Test.txt");
                        if(!in)       {
                        cout << "Failed to open file" << endl;
                        return 1;  }
            char c;
            in.ignore(6 , ' ');
            while(in)  {
                        in.get(c);
                        cout << c;}
            return 0;
}
```


Test - Notepad
File Edit Format View Help
Ali 100
Abid 200


C:\Users\Administrator\Downloads\Untitled1.exe
100
Abid 200

Process exited after 0.0147 seconds with return value 0
Press any key to continue . . .

```
#include <iostream>
#include <fstream>
using namespace std;
int main(){
ifstream in("Test.txt");
                    if(!in)       {
                    cout << "Failed to open file" << endl;
                    return 1;  }
          char c;
          in.ignore(5 , 'i');
          while(in)  {
                    in.get(c);
                    cout << c;}
          return 0;
}
```


C:\Users\Administrator\Downloads\Untitled1.exe
100
Abid 200

Process exited after 0.01905 seconds with return value 0
Press any key to continue . . .

C++ Binary *read()* and *write()* Functions

- Write an object of a class to this file, by using the *write()* function.
- **write((char *) & ob, sizeof(ob));**
- Read the stored object from the file, by using the *read()* function.
- **read((char *) & ob, sizeof(ob));**

C++
Binary *read()* and *write()* Functions

Binary I/O Functions	Description
read()	This binary function is used to perform *file input operation* i.e. to read the objects stored in a file.
write	This binary function is used to perform *file output operation* i.e. to write the objects to a file, which is stored in the computer memory in a **binary form**. *Only the data member of an object are written and not its member functions*

File pointers

- Every file maintains two pointers called get_pointer (in input mode file) and put_pointer (in output mode file) which tells the current position in the file where reading or writing will takes place. These pointers help attain random access in file. That means moving directly to any location in the file instead of moving through it sequentially.

- There may be situations where random access in the best choice. For example, if you have to modify a value in record no 21, then using random access techniques, you can place the file pointer at the beginning of record 21 and then straight-way process the record. If sequential access is used, then you'll have to unnecessarily go through first twenty records in order to reach at record 21.

Function	Description
seekg( )	Moves the get pointer to a specific location in the file
seekp( )	Moves the **put** pointer to a specific location in the file
tellg( )	Returns the position of **get** pointer
tellp( )	Returns the position of **put** pointer

The seekg(), seekp(), tellg() and tellp() Functions

- random access is achieved by manipulating seekg(), seekp(), tellg() and tellp() functions. The seekg() and tellg() functions allow you to set and examine the get_pointer, and the seekp() and tellp() functions perform these operations on the put_pointer.
- The seekg() and tellg() functions are for input streams (ifstream) and seekp() and tellp() functions are for output streams (ofstream).

tellg() and tellp()

- The functions tellg() and tellp() return the position, in terms of byte number, of put_pointer and get_pointer respectively, in an output file and input file.

Random Access

- We can move the pointer (while reading or writing) to a specific position in a file

istream &seekg(int *offset, origin);

ostream &seekp(int *offset, origin);

Where origin can be any of the three following options

Origin Options

- ios::beg *// Beginning-of-file position*
- ios::cur *// Current location position*
- ios::end *// End-of-file position*

If we don't specify the second argument, then ios :: beg is the default reference position.

- fin.seekg(30); // will move the get_pointer (in ifstream) to byte number 30 in the file(starts from beg)
- fout.seekp(30);// will move the put_pointer (in ofstream) to byte number 30 in the file
- It automatically points at the beginning of file, allowing us to read the file from the beginning.
- fin.seekg(30, ios::beg); // go to byte no. 30 from beginning of file
- fin.seekg(-2, ios::cur); // back up 2 bytes from the current position
- fin.seekg(0, ios::end); // go to the end of the file
- fin.seekg(-4, ios::end); // backup 4 bytes from the end of the file
-

seekg() Function

- The **seekg( )** function moves the associated file's current pointer *offset the number* of characters from the specified *origin*

- The **seekg()** function is member of the *ifstream* class and is called through an *ifstream* object

- The function only moves the pointer ahead, the reading operation should then be performed through some other function

Example

```
// Starting from the beginning, move position
   pointer five characters further

i.seekg(5, ios::beg);
char chR[20];
i.read(chR, 20);
for(int i = 0; i < 20; i++)
{
        cout << chR[i];
}
```

seekp() Function

- The **seekp()** function moves the associated file's current pointer *offset the number* of characters from the specified *origin*

- The **seekp()** function is member of the *ofstream* class and is called through an *ofstream* object

- The function only moves the pointer ahead, the writing operation should then be performed through some other function

Example 1

// Starting from the beginning, move position pointer two characters forward… then perform writing with put()

```
char ch = 'K';
i.seekp(2, ios::beg);
i.put(ch);   // writes K at third position in the file
```

Example 2

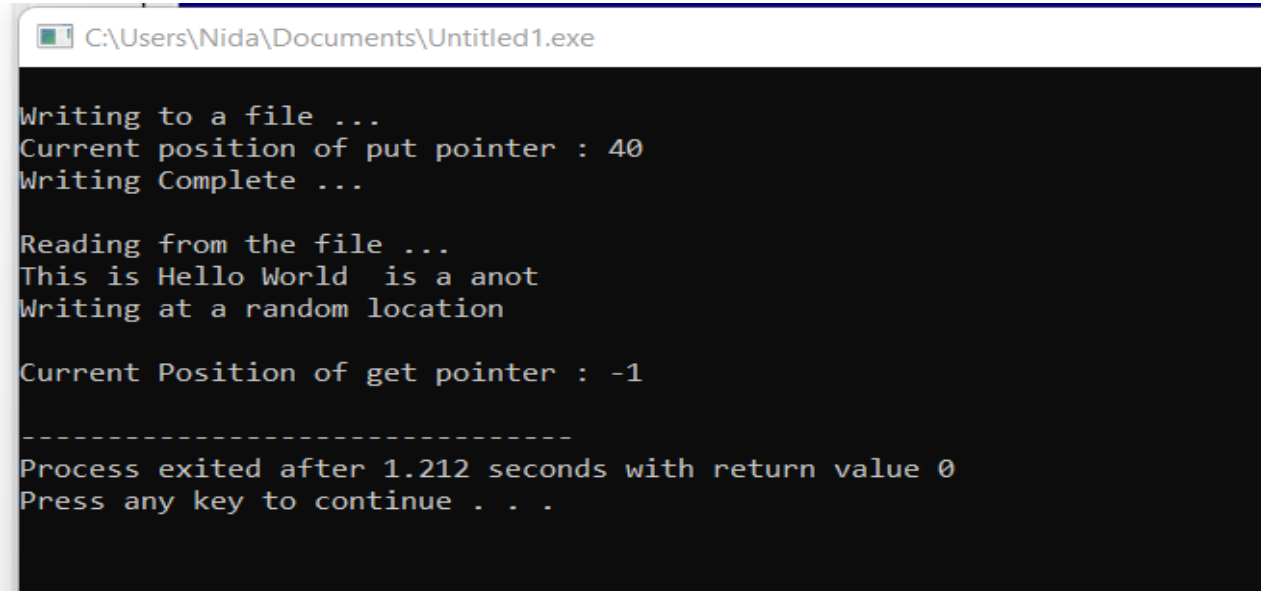
// Starting from the end, move position pointer three characters backwards… then perform writing with put()

```
char ch = 'J';
i.seekp(-3, ios::end);
i.put(ch);   // writes J at fourth from last position
```

```
#include<iostream>
#include<fstream>
using namespace std;
int main() {
  fstream fp;
  char buf[100];
  int pos;
  // open a file in write mode with 'ate' flag (ate to move the pointer)
  fp.open("pointer.txt", ios :: out | ios :: ate);
  cout << "\nWriting to a file ... " << endl;
  fp << "This is a line" << endl; // write a line to a file
  fp << "This is a another line" << endl; // write another file
  pos = fp.tellp();
  cout << "Current position of put pointer : " << pos << endl;
```

```
// move the pointer 10 bytes backward from current position
   fp.seekp(-10, ios :: cur);
   fp << endl << "Writing at a random location  ";
   // move the pointer 7 bytes forward from beginning of the file
   fp.seekp(7, ios :: beg);
   fp << " Hello World  ";
   fp.close(); // file write complete
   cout << "Writing Complete ... " << endl;
```

```
// open a file in read mode with 'ate' flag
  fp.open("pointer.txt", ios :: in | ios :: ate);
  cout << "\nReading from the file ... " << endl;
  fp.seekg(0); // move the get pointer to the beginning of the file
  // read all contents till the end of file
  while (!fp.eof()) {
    fp.getline(buf, 100);
    cout << buf << endl;
  }
  pos = fp.tellg();
  cout << "\nCurrent Position of get pointer : " << pos << endl;
  return 0;
}
```


C:\Users\Nida\Documents\Untitled1.exe
Writing to a file ...
Current position of put pointer : 40
Writing Complete ...
Reading from the file ...
This is Hello World is a anot
Writing at a random location
Current Position of get pointer : -1

Process exited after 1.212 seconds with return value 0
Press any key to continue . . .